

awesome—let's lock down the **requirements**, **pseudocode**, **CLI**, and **tests** so Claude can generate `mail_to_rag.py` and extend your `cli.py` cleanly. I'll keep it opinionated, idempotent, and 3-step (import → clean/Q → A → chunk).

Functional requirements

1. Source & storage

- Read one or more `.mbox` files from `$PROJECT_ROOT/data/email/`.
- Use **SQLite** (single DB file, default `qrplabs.db` at project root).
- Idempotent: safe to re-run without duplicating rows.

2. Tables (single DB)

- `RawEmail`(`raw_id` PK, `message_id`, `thread_hint`, `date_hdr`, `from_addr`, `to_addrs`, `subject`, `headers`, `body_raw`)
- `CleanEmail`(`clean_id` PK, `thread_id`, `message_id`, `from_addr`, `subject`, `msg_date`, `text_clean`, `is_question` INTEGER, `imported_to_rag_at` TEXT NULL)
- `QAPair`(`qa_id` PK, `thread_id`, `question_id` FK→`CleanEmail`, `answer_id` FK→`CleanEmail`, `created_at` TEXT)
- `RagChunk`(`chunk_id` PK, `qa_id` FK→`QAPair`, `thread_id`, `question_id`, `answer_id`, `pos` INT, `text`, `tokens_est` INT, `created_at` TEXT)

3. Deterministic IDs

- `raw_id` = `sha1(Message-ID || Date || Subject)` (use first non-empty; fallback concatenation)
- `thread_id` = `sha1(earliest_date_in_thread_hint || normalized_subject)`
- `qa_id` = `sha1(thread_id || question.clean_id || answer.clean_id)`
- `chunk_id` = `sha1(qa_id || chunk_index)`

4. Cleaning & pairing

- Cleaning: strip quoted lines beginning with `>`, collapse blank lines, remove common “wrote:” separators, trim.
- Question detection: `?` in text **or** starts with common interrogatives (`how/what/why/can/does/do/is/are/should/could/would`).
- Q → A: For each message (by thread/date), link to nearest previous **question**; fallback = nearest previous **different sender**.

5. Chunking for RAG

- Default chunk is **Q → A concatenation** (“Question:\n...\n\nAnswer:\n...”).
- Split into **~300–500 tokens** (configurable) with **10–20% overlap**.
- Token approximation allowed (chars/4), but design for pluggable tokenizer.

6. Idempotency

- `INSERT OR IGNORE / INSERT OR REPLACE` as appropriate.
- Only create chunks for `QAPair` rows not already present.
- Set `imported_to_rag_at` on `CleanEmail` rows touched by chunking.

7. Configurability

- CLI flags for: database path, input file(s), purge behavior for `RawEmail`, chunk sizes/overlap, dry-run, and verbose logging.

Non-functional requirements

- **Python 3.8+**, PEP8, docstrings.
- No network calls; all local/offline.
- Robust to odd messages (skip on parse error, log warning).
- Clear exit codes (0 success, 2 usage error, 1 partial failures).

Directory expectations

```
$PROJECT_ROOT/  
  data/email/           # .mbox files live here (user-provided)  
  src/<your_pkg>/  
    cli.py              # will be extended  
    mail_to_rag.py      # new module Claude will write  
  tests/  
    data/mbox/          # test fixtures (tiny .mbox)  
    test_mail_to_rag.py
```

CLI design (commands & args)

Main verb: `mail`

```
Usage:  
  qmx-helpdesk mail import  --mbox <path|glob> [--db <sqlite>] [--purge-  
raw] [--verbose]  
  qmx-helpdesk mail clean  [--db <sqlite>] [--verbose]  
  qmx-helpdesk mail chunk  [--db <sqlite>] [--max-tokens 450] [--overlap  
50] [--verbose]
```

```
qmx-helpdesk mail all --mbox <path|glob> [--db <sqlite>] [--purge-raw] [--max-tokens 450] [--overlap 50] [--verbose] [--dry-run]
```

Help (verbose)

- `import`: Loads `.mbox` → `RawEmail`. Optionally `--purge-raw` to clear `RawEmail` first. Safe to re-run.
- `clean`: Produces `CleanEmail` (1:1 with `RawEmail`) and `QAPair` (thread-aware links). Idempotent upsert.
- `chunk`: Builds `RagChunk` from new `QAPair`s, then stamps `CleanEmail.imported_to_rag_at`.
- `all`: Convenience run of all 3 with one command.

Arguments:

- `--mbox`: file path or glob (e.g., `data/email/Category_Forums*.mbox`).
- `--db`: path to sqlite db (default: `qrplabs.db`).
- `--purge-raw`: delete from `RawEmail` before importing.
- `--max-tokens` / `--overlap`: chunk parameters.
- `--dry-run`: compute but don't mutate DB for `all` (log what would change).
- `--verbose`: INFO-level logging.

Pseudocode (module `mail_to_rag.py`)

```
MODULE mail_to_rag

CONST DEFAULT_DB = "qrplabs.db"

function sha1(str) -> hex

function now_iso() -> str (UTC ISO8601 Z)

function norm_subject(subj: str) -> str
    strip "re:", "fwd:", etc.
    lower, trim

function has_question(text: str) -> bool
    if "?" in text: return True
    if startswith any of
        ["how", "what", "why", "can", "does", "do", "is", "are", "should", "could", "would"]:
```

```
        return True
    return False
```

```
function strip_quotes_and_noise(text: str) -> str
    remove lines starting with ">"
    remove common "---- wrote:" blocks
    collapse 3+ newlines → 2
    trim
```

```
function approx_token_len(s: str) -> int
    return max(1, len(s) // 4)
```

```
function chunk_text(s: str, max_tokens: int, overlap_tokens: int) ->
list[str]
    max_chars = max_tokens * 4
    ov_chars = overlap_tokens * 4
    initialize chunks = []
    i = 0
    while i < len(s):
        j = min(len(s), i + max_chars)
        # try sentence boundary search; else j
        k = best_boundary_between(i, j) or j
        chunks.append(s[i:k].strip())
        if k >= len(s): break
        i = max(k, i + max_chars - ov_chars)
    return non-empty chunks
```

```
SCHEMA (execute once on connection):
    CREATE TABLE IF NOT EXISTS RawEmail (...)
    CREATE INDEXes...
    CREATE TABLE IF NOT EXISTS CleanEmail (...)
    CREATE TABLE IF NOT EXISTS QAPair (...)
    CREATE TABLE IF NOT EXISTS RagChunk (...)
    INDEXes...
```

```
function ensure_schema(db_path)
```

```
# STEP 1: IMPORT
```

```
function import_mbox(db_path, mbox_paths: list[str], purge_raw: bool,
logger)
    ensure_schema(db_path)
    if purge_raw: DELETE FROM RawEmail
    for each path in mbox_paths:
        open mailbox.mbox(path)
        for msg in mbox:
            read headers: Message-ID, Subject, Date, From, To
```

```

        compute headers_text up to blank line
        extract body_raw:
            if multipart: prefer text/plain; fallback text/html → strip
tags
            else: decode payload using charset or utf-8; fallback
str(payload)
        thread_hint = norm_subject(Subject)
        key = Message-ID or (Date + Subject)
        raw_id = sha1(key)
        INSERT OR REPLACE INTO RawEmail(...)
    commit

```

STEP 2: CLEAN + PAIR

```

function derive_thread_id(rows_in_same_hint) -> str
    earliest_date = min(date_hdr)
    return sha1(earliest_date + thread_hint)

```

```

function clean_and_pair(db_path, logger)

```

```

    ensure_schema
    SELECT * FROM RawEmail ORDER BY thread_hint, date_hdr
    group by thread_hint
    for each group:
        thread_id = derive_thread_id(group)
        for each row r in group:
            clean_id = r.raw_id
            text_clean = strip_quotes_and_noise(r.body_raw)
            is_q = has_question(text_clean) ? 1 : 0
            INSERT OR IGNORE INTO CleanEmail(clean_id, thread_id,
message_id, from_addr, subject, msg_date, text_clean, is_question,
imported_to_rag_at=NULL)

```

```

    # build QAPair
    SELECT clean rows ORDER BY thread_id, msg_date
    group by thread_id
    for each thread list:
        for i from 1..n-1:
            answer = msgs[i]
            find nearest prior j < i where msgs[j].is_question == 1
            if none, find nearest prior with different sender
            if still none, continue
            qa_id = sha1(thread_id + question.clean_id + answer.clean_id)
            INSERT OR IGNORE INTO QAPair(qa_id, thread_id, question_id,
answer_id, created_at=now_iso)
    commit

```

STEP 3: CHUNK

```

function chunk_for_rag(db_path, max_tokens, overlap, logger)

```

```

ensure_schema
SELECT q.qa_id, q.thread_id, q.question_id, q.answer_id, mq.text_clean,
ma.text_clean
FROM QAPair q
JOIN CleanEmail mq ON mq.clean_id = q.question_id
JOIN CleanEmail ma ON ma.clean_id = q.answer_id
LEFT JOIN RagChunk rc ON rc.qa_id = q.qa_id
GROUP BY q.qa_id
HAVING COUNT(rc.qa_id) = 0    # only new pairs
for each row:
    base_text = ""
    if q_text: base_text += "Question:\n" + q_text + "\n\n"
    base_text += "Answer:\n" + a_text
    parts = chunk_text(base_text, max_tokens, overlap)
    for idx, part in enumerate(parts):
        chunk_id = sha1(q.qa_id + "|" + str(idx))
        tokens_est = approx_token_len(part)
        INSERT OR IGNORE INTO RagChunk(chunk_id, qa_id, thread_id,
question_id, answer_id, pos=idx, text=part, tokens_est, created_at=now_iso)
        UPDATE CleanEmail SET imported_to_rag_at = now_iso WHERE clean_id IN
(question_id, answer_id) AND imported_to_rag_at IS NULL
    commit

```

cli.py extension (proposed help text excerpts)

```

qmx-helpdesk mail
    Import and prepare forum emails for retrieval and analytics.

```

Commands:

```

import    Load one or more .mbox files into the RawEmail table.
clean     Produce cleaned messages and Q→A links (thread-aware).
chunk     Build RAG-friendly chunks from Q→A pairs.
all       Run import → clean → chunk in one go.

```

Examples:

```

qmx-helpdesk mail import --mbox "data/email/Category_Forums*.mbox" --db
qrplabs.db --purge-raw
qmx-helpdesk mail clean --db qrplabs.db
qmx-helpdesk mail chunk --db qrplabs.db --max-tokens 450 --overlap 50
qmx-helpdesk mail all --mbox "data/email/Category_Forums*.mbox" --db
qrplabs.db --purge-raw --max-tokens 450 --overlap 50

```

Options:

```
--mbox <path|glob>    One or more .mbox files (glob allowed). Required for
'import' and 'all'.
--db <path>           SQLite DB path (default: qrplabs.db).
--purge-raw           Before import, delete existing rows from RawEmail.
--max-tokens <int>    Chunk size in tokens approximation (default: 450).
--overlap <int>       Overlap in tokens between chunks (default: 50).
--dry-run             Compute without DB mutation (for 'all').
--verbose             Verbose logging.
```

Unit tests (strategy + fixtures)

Test data fixture (checked into repo)

Create a tiny deterministic `.mbox` with **one short thread** ($Q \rightarrow A \rightarrow$ follow-up) and **one unrelated message**. Save in `tests/data/mbox/qrplabs_small.mbox`.

How Claude can generate it (Python snippet to run once locally; commit the resulting file):

```
import mailbox, email, time, os

fn = "tests/data/mbox/qrplabs_small.mbox"
os.makedirs(os.path.dirname(fn), exist_ok=True)
mb = mailbox.mbox(fn, create=True)

def add_msg(subject, from_, to, date, msg_id, in_reply_to=None,
references=None, body=""):
    m = email.message.EmailMessage()
    m['Subject'] = subject
    m['From'] = from_
    m['To'] = to
    m['Date'] = date
    m['Message-ID'] = msg_id
    if in_reply_to: m['In-Reply-To'] = in_reply_to
    if references: m['References'] = references
    m['X-Gmail-Labels'] = "Category:Forums,Inbox"
    m.set_content(body)
    mb.add(m)

# Thread 1: Q -> A -> follow-up (non-question)
t = time.gmtime(1724832000) # fixed time
d1 = time.strftime("%a, %d Aug %Y 10:00:00 +0000", t)
d2 = time.strftime("%a, %d Aug %Y 10:10:00 +0000", t)
d3 = time.strftime("%a, %d Aug %Y 10:20:00 +0000", t)
```

```

q_id = "<q1@example>"
a_id = "<a1@example>"
f_id = "<f1@example>"

add_msg("[QRPLabs] SWR too high on 20m", "joe <k7abc@example.com>",
"list@example.com", d1, q_id, body="My SWR is 3:1 on 20m. Any ideas?")
add_msg("Re: [QRPLabs] SWR too high on 20m", "alice <wlxyz@example.com>",
"list@example.com", d2, a_id, in_reply_to=q_id, references=q_id, body="Add a
common-mode choke on the feedline.\n> quoted stuff...")
add_msg("Re: [QRPLabs] SWR too high on 20m", "joe <k7abc@example.com>",
"list@example.com", d3, f_id, in_reply_to=a_id, references="%s %s" % (q_id,
a_id), body="Thanks!")

# Thread 2: unrelated, no question
u_id = "<u1@example>"
add_msg("[QRPLabs] Build report", "tom <k7tom@example.com>",
"list@example.com", d1, u_id, body="Built QCX-mini successfully.")

mb.flush()
mb.close()
print("wrote", fn)

```

Commit the generated `qrplabs_small.mbox` so tests are deterministic.

Suggested tests (`tests/test_mail_to_rag.py`)

1. Import creates RawEmail rows

- Run `import_mbox` on the fixture.
- Assert: number of rows = 4, no duplicates, `thread_hint` normalized (strip "re:").
- Assert: `body_raw` present.

2. Clean creates CleanEmail 1:1 and sets `is_question`

- Run `clean_and_pair`.
- Assert: `CleanEmail` count == `RawEmail` count.
- Assert: the Q message has `is_question` = 1, others 0.

3. Thread id deterministic

- Assert: all three messages from Thread 1 share the same `thread_id`; Thread 2 has a different one.

4. QAPair links nearest previous question

- Assert: One `QAPair` exists linking Q → A (joe → alice).
- Follow-up "Thanks!" should **not** create a Q → A (unless you want it to); assert pair count == 1.

5. Chunking produces RagChunk rows

- Run `chunk_for_rag(max_tokens=60, overlap=10)` on small text.
- Assert: at least 1 chunk for that QAPair; `chunk.text` contains “Question:” and “Answer:”.
- Assert: `CleanEmail.imported_to_rag_at` set for Q and A (not for unrelated message).

6. Idempotency

- Re-run all 3 steps a second time; counts remain stable (no growth).

7. CLI glue (optional integration test)

- Use `subprocess` to run `qmx-helpdesk mail all --mbox tests/data/mbox/qrplabs_small.mbox --db :memory:` (or temp file).
- Assert final table counts.

Verbose help text (drop into `cli.py`)

- For each subcommand, include:
 - Purpose
 - What tables it touches
 - Idempotency note
 - Examples (with `data/email/*.mbox`)
 - Defaults (db path, token sizes)

Stretch ideas (optional later)

- Parse `Message-ID` / `In-Reply-To` / `References` into explicit columns for stronger threading.
- Add `messages_fts` (SQLite FTS5) for BM25 hybrid search.
- Allow `--experts-file` (newline emails/callsigns) to mark `is_expert` on answers.
- Plug in a real tokenizer (tiktoken) if you standardize on a model.

if you give Claude this spec, it can generate `mail_to_rag.py`, extend `cli.py` with the subcommands + detailed help, and create the test fixture and unit tests exactly as outlined.